

Exemplo – Torre de Hanói

O próximo exemplo é uma aplicação significativamente mais complexa de pilhas. Ele está relacionado ao jogo matemático Torre de Hanói. Vamos começar com as regras. O jogo requer três hastes, sobre as quais você pode colocar discos. Cada disco tem um tamanho diferente. No início, todos os discos são colocados na primeira haste, formando a pilha, ordenados do menor (no topo) ao maior (na base).

O objetivo do jogo é mover todos os discos da primeira haste (FROM) para a segunda (TO). No entanto, durante todo o jogo, você não pode colocar um disco maior sobre um menor. Além disso, você só pode mover um disco por vez e, claro, só pode retirar um disco do topo de qualquer haste. O problema pode ser dividido em subproblemas.

Se houver apenas um disco, basta movê-lo de FROM para TO, sem usar a haste AUXILIARY. Se houver dois discos, mova o primeiro de FROM para AUXILIARY, depois o maior de FROM para TO, e por fim o menor de AUXILIARY para TO. Com três discos, primeiro mova dois discos de FROM para AUXILIARY (usando TO como auxiliar), depois mova o disco maior de FROM para TO e, por fim, mova os dois discos de AUXILIARY para TO (usando FROM como auxiliar).

De forma geral, para mover n discos: primeiro mova $n-1$ discos de FROM para AUXILIARY (usando TO como auxiliar), em seguida mova o último disco de FROM para TO, e por fim mova $n-1$ discos de AUXILIARY para TO (usando FROM como auxiliar).

Agora que você conhece as regras básicas, vamos ao código.

Primeiro, a classe HanoiTower contém a lógica do jogo:

```
public class HanoiTower
{
    public int DiscsCount { get; private set; }
    public int MovesCount { get; private set; }
    public Stack<int> From { get; private set; }
    public Stack<int> To { get; private set; }
    public Stack<int> Auxiliary { get; private set; }
    public event EventHandler<EventArgs> MoveCompleted; (...)
}
```

A classe possui cinco propriedades: o número de discos (DiscsCount), o número de movimentos realizados (MovesCount) e as três hastes (From, To, Auxiliary). O evento MoveCompleted é disparado após cada movimento, permitindo atualizar a interface gráfica.

O construtor recebe o número de discos e inicializa as pilhas:

```
public HanoiTower(int discs)
{
    DiscsCount = discs;
    From = new Stack<int>();
    To = new Stack<int>();
    Auxiliary = new Stack<int>();
    for (int i = 1; i <= discs; i++)
    {
        int size = discs - i + 1;
        From.Push(size);
    }
}
```

O método Start inicia o algoritmo chamando o método recursivo Move, que realiza os movimentos conforme as regras descritas.

```
public void Start()
{
    Move(DiscsCount, From, To, Auxiliary);
}
```

O método Move é chamado recursivamente para mover os discos entre as hastes.

```
public void Move(int discs, Stack<int> from, Stack<int> to, Stack<int> auxiliary)
{
    if (discs > 0)
    {
        Move(discs - 1, from, auxiliary, to);
        to.Push(from.Pop());
        MovesCount++;
        MoveCompleted?.Invoke(this, EventArgs.Empty);
        Move(discs - 1, auxiliary, to, from);
    }
}
```

Esse processo garante que todos os discos sejam movidos de FROM para TO, seguindo as regras do jogo Torre de Hanói.